设计mq

背景

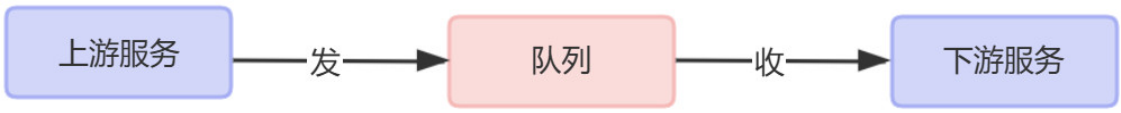
两个不同的服务交互, 一般都是发rpc请求.

- 但是这样就发生了强耦合, 一个服务的变更会直接影响依赖它的所有上下游服务, 牵一发而动全身.
- 并且rpc的同步调用还比较慢, 上游服务需等待所有下游服务执行完成才能返回结果.

队列模型

那有没有这样一种可能, 上游服务, 比如说订单服务, 只需要发一条通知, 说下单成功了, 几个不同的下游服务, 比如支付服务, 商品服务, 通知服务就能直接收到消息, 然后自动扣减库存, 自动发起支付, 自动发送通知.

说到这儿, 想到了什么? 队列. 上游服务发送消息到队列, 下游服务监听队列消息.

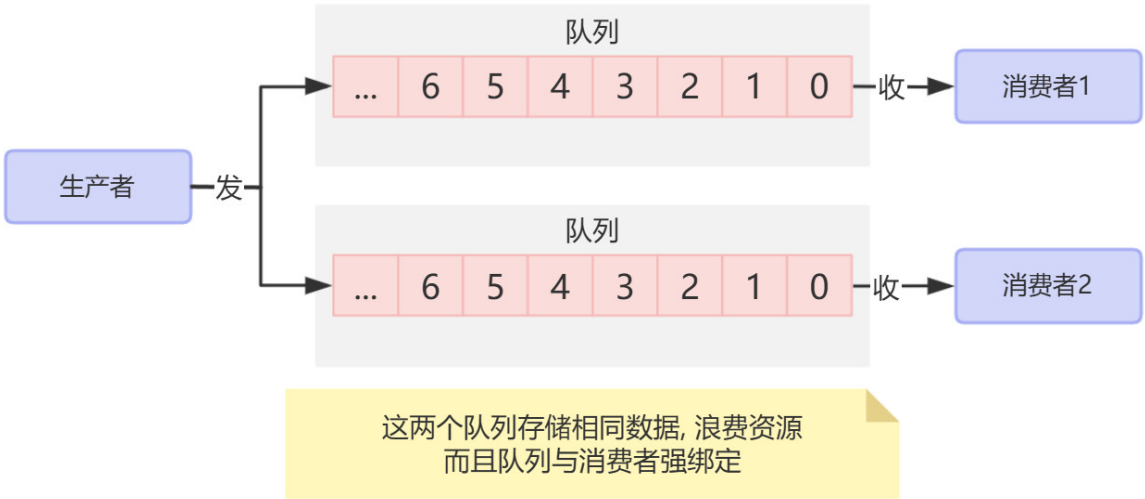


此时上游服务就是一个消息的生产者, 下游服务就是一个消息的消费者.

这个叫做队列模型, 它的特点是点对点消费. 就是消息被发送到队列, 多个消费者竞争消费, 每个消息仅被一个消费者处理.

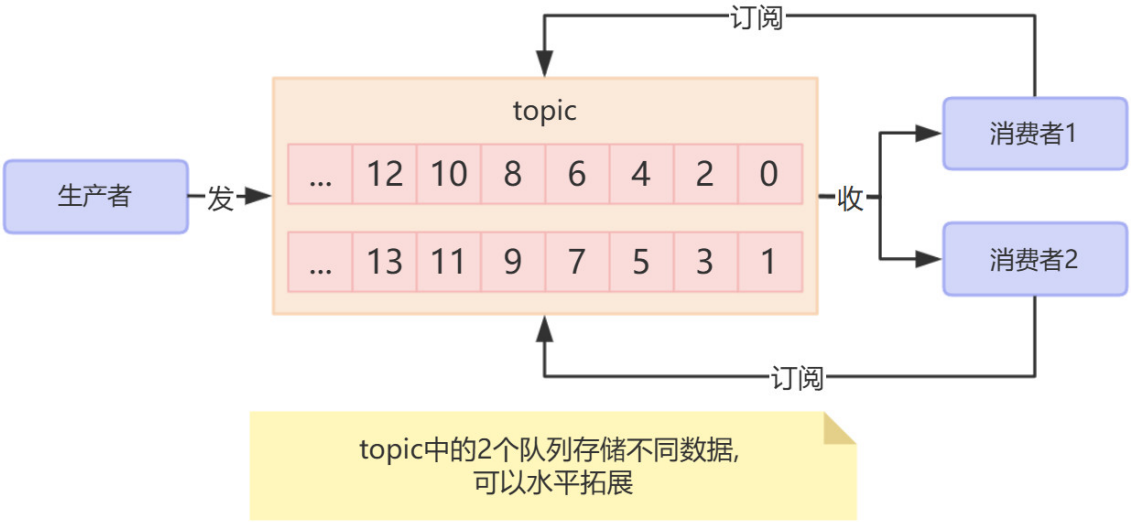
如果有多个消费者想同时消费同一个消息, 那就只能复制多个队列, 每个消费者都消费自己的队列. 但是复制多个队列不仅浪费资源, 而且队列和消费者强绑定了, 某个消费者不消费了, 就要去掉对应的队列. 这完全违背了解藕的目标.

主题模型



队列模型不太好, 那怎么办呢?

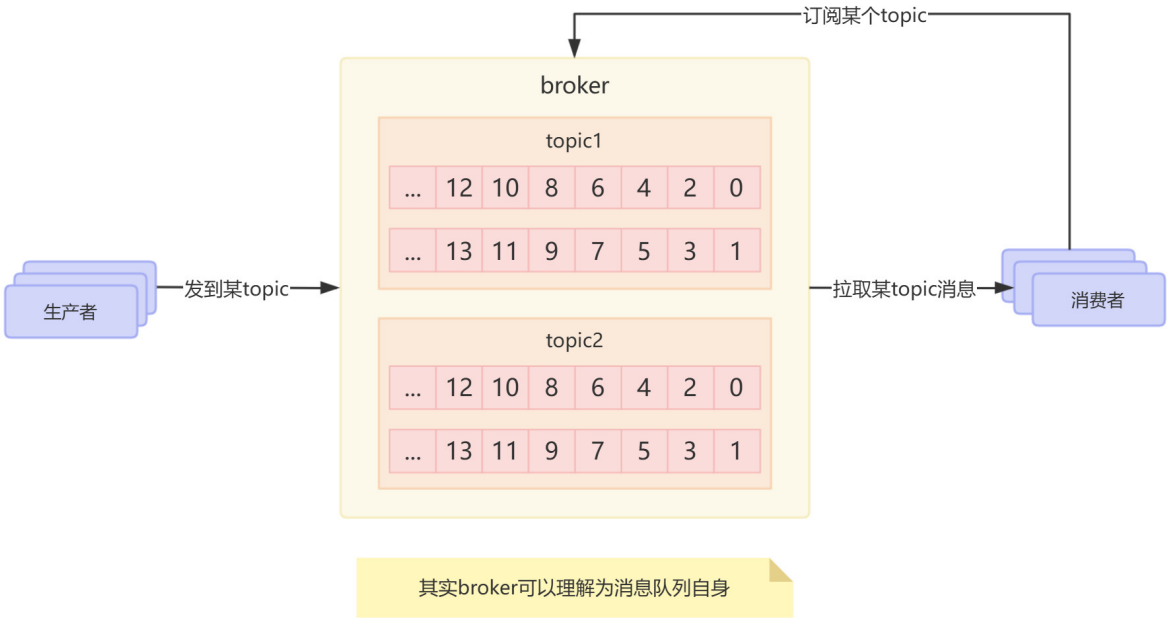
- 我们可以让每个消费者订阅某个主题, 也就是topic
- 生产者也是发送消费到这个topic, 那队列就可以把收到的消息发送给所有订阅了这个topic的消费者.
- 这个就是主题模型, 消费者不再依赖队列, 而是依赖topic, 一个topic中可以有一个或者多个队列, 这样队列可以随意地水平扩容.



那给我们发明的这个玩意儿起个名吧. 名字好难想啊. 既然是一个能收发消息的队列, 那不如就叫它消息队列如何.

broker

消息队列中不可能只有一个topic, 可能有多个topic, 所以得在外面包一层来管理多个topic. 那管理topic的这玩意儿就叫它broker吧. 生产者把消息发送到broker, broker根据消息的topic来把消息放到对应的队列中, 然后消费者根据topic获取消息.



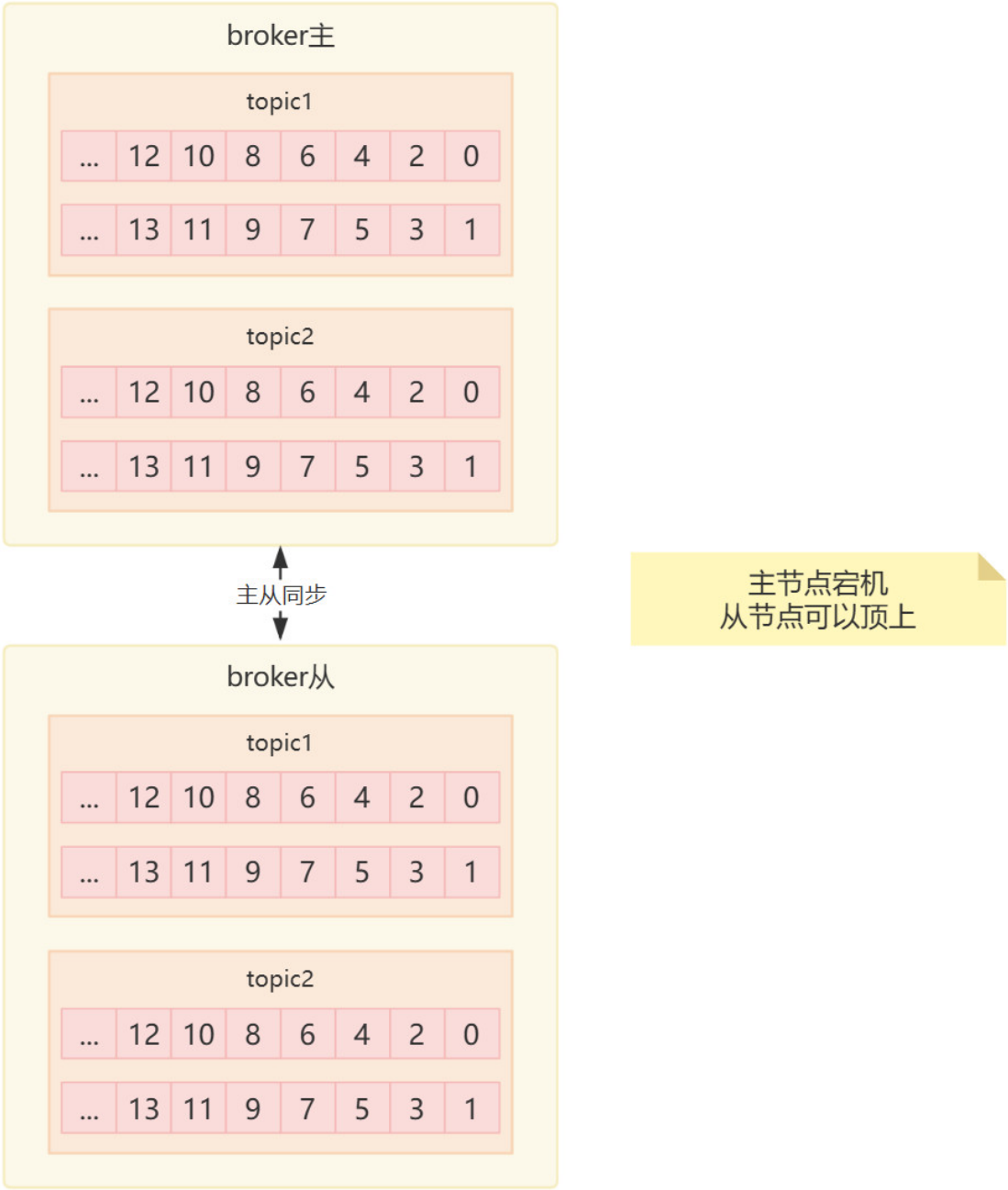
那么问题又来了, 把消息存到内存里不是很安全, 重启就会丢失, 所以Broker收到消息后需要将收到的消息存到磁盘中, 做持久化.

主从

那么问题又来了, 如果你broker只有一个, 那就会有单点故障问题, 所以起码得做个主从吧.

- broker主节点用来读写, 从节点作为备份
- 从节点定期从主节点中同步数据.

想想mysql的读写分离, redis的主从架构, 是不是非常相似.

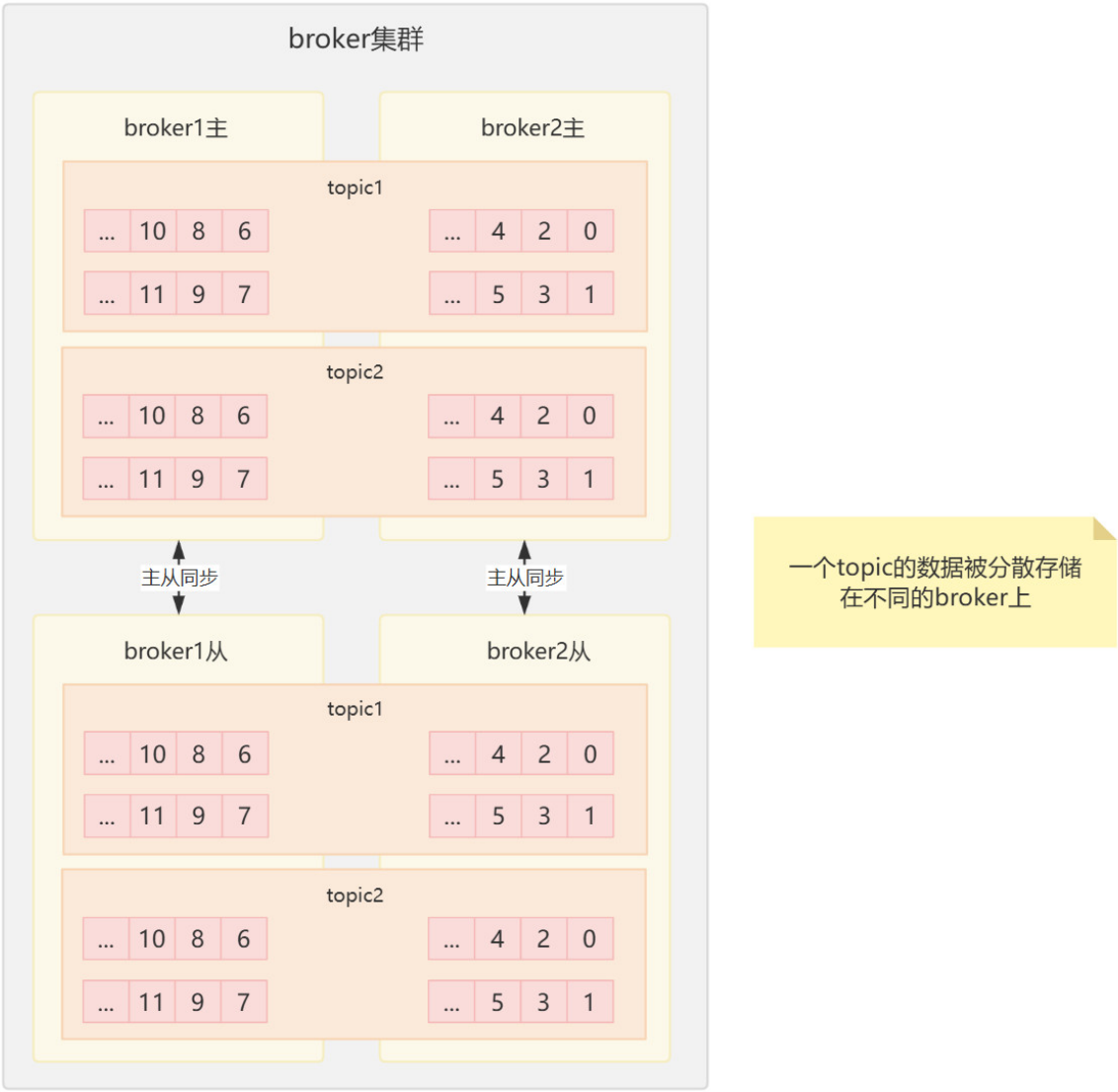


集群

那么问题又来了, broker主从存的数据都是一样的, 那每个broker所能承载的消息数量也是有上限的, 而且主节点并发写的能力也是有瓶颈的. 怎么办呢?

- 那就部署多个主从结构, 构成一个broker集群
- 把每个topic中的队列和数据分散存储在不同的broker节点上.
- 这样集群中的不同主节点都各自保存topic的部分数据, 然后每个节点的压力都减少了.

想想mysql的分库分表, redis的分片集群是不是非常相似.



注册中心

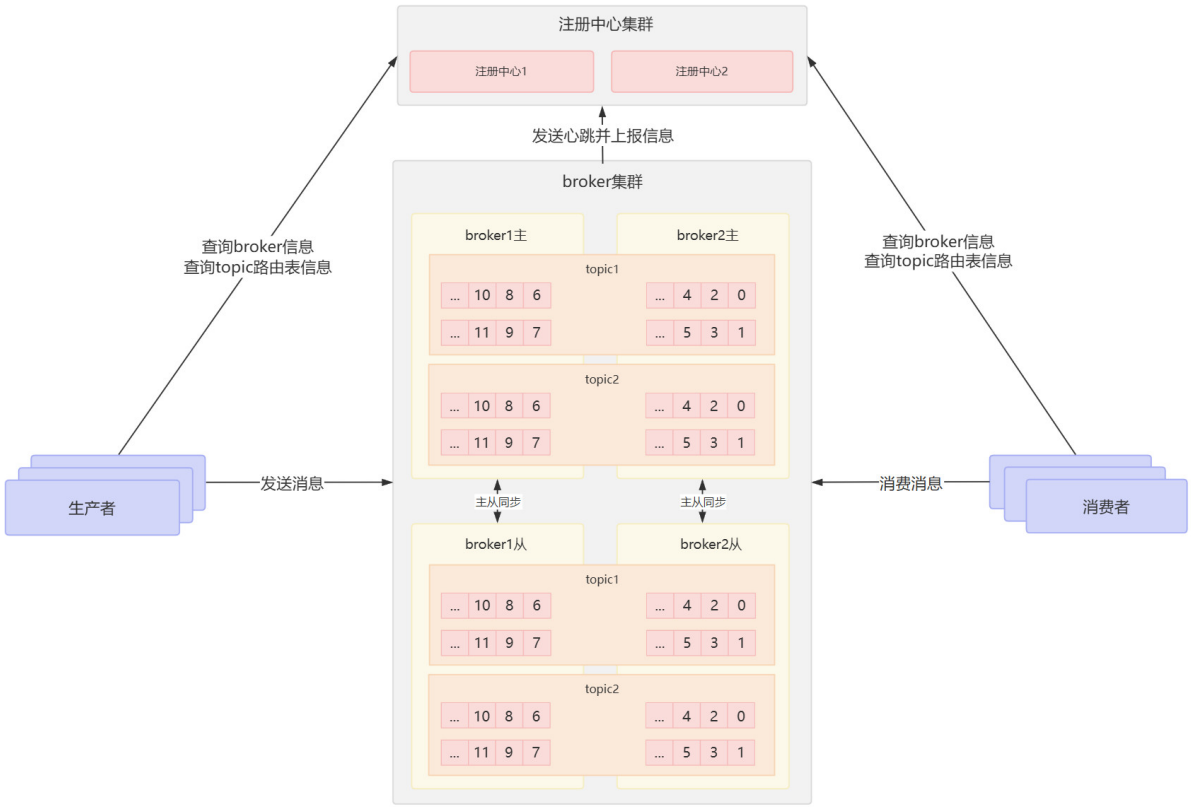
那么问题又来了, 那么多个broker, 生产者发消息的时候, 怎么知道应该发给哪个broker呢, 消费者获取消息的时候, 怎么知道应该从哪个broker中获取消息呢?

所以这时候就需要一个注册中心, 来专门管理broker集群.

这个注册中心说白了就这么几个作用.

- 第一个保存集群状态信息, 通过broker定时上报心跳, 来实时感知节点上下线, 确保请求不发到故障节点.
- 第二个, 掌管消息路由, 让生产者知道消息应该发给哪个broker, 让消费者知道应该从哪个broker上取消息.
- 第三个负责集群负载均衡, 故障转移.

那为了保证注册中心的高可用, 注册中心自身也得做集群.



工作流程

所以整个消息队列的工作流程就是这样的.

- broker集群定时向注册中心发送心跳包, 上报自身信息.
- 注册中心掌管了所有broker节点信息和消息的路由信息.
- 生产者要给某个topic发送消息, 通过访问注册中心就能获取到某个topic的路由表, 就知道自己的消息应该发送给哪个broker的哪个队列, 然后对应broker主节点收到消息后同步消息给从节点, 并将消息持久化.
- 消息的消费者访问注册中心, 订阅对应的topic, 就能获取到这个topic的路由表, 就能知道应该去哪个broker的哪个队列拉取消息了.

看到这儿, 恭喜你, 你发明了消息队列.